

Technical Work-Test

RAG-Powered Conversational Learning Assistant

A practical assessment for the Data Science / AI Engineer role

ROLE	Data Science / AI Engineer - RAG & LLM systems
SUGGESTED EFFORT	6–8 focused hours, across up to 1 calendar day
DELIVERABLE	Working prototype, test suite, and a short written rationale
STACK	Your choice - we assess judgement and engineering, not a framework

Confidential - issued to the named candidate only.

Contents

0 Before you start	3
1 Context	4
1.1 What we will provide	4
1.2 The 31 sub-facets	5
2 The agreed chat flow	6
2.1 Entry point	6
2.2 Consent and report	6
2.3 Select an area - and hold it	6
2.4 Brief coaching conversation	6
2.5 Recommend a Learning Pathway	6
2.6 Resume branch	6
3 Technical constraints	8
3.1 Latency	8
3.2 Retrieval and grounding	8
3.3 Cost and scale	8
3.4 State and memory	8
4 What to deliver	9
4.1 Required	9
4.2 Nice to have, only if time allows	9
4.3 Explicitly out of scope	9
5 Test and edge cases	10
5.1 Happy path	10
5.2 Edge cases	10
5.3 Adversarial and safety	10
6 How we score it	12

0 Before you start

This work-test is built around a realistic problem rather than an artificial puzzle, so your time goes into something representative of the work. There is no single correct solution. We are interested in how you scope an ambiguous brief, make and defend engineering trade-offs, and produce something that runs.

What we assess

- RAG engineering - chunking, retrieval quality, grounding, and a clean "no answer in corpus" path.
- Conversation design - turning a messy specification into a predictable state machine.
- Systems thinking - latency budgets, caching, cost awareness, and graceful failure.
- Testing rigour - coverage across happy paths, edge cases and adversarial input.
- Communication - a clear README and an honest account of what you did and did not do.

Scope control is itself part of the assessment. We do not expect a production system in the suggested time. We expect a working core, sound reasoning about the remainder, and honest notes on anything stubbed or deferred. A smaller piece done well is stronger than a large piece left half-working.

Use of AI tooling

You are welcome to use AI coding assistants - we use them daily. The condition is that you understand and can defend every line you submit; we will walk through your code together on a follow-up call.

1 Context

One of our products is an in-app AI learning companion (persona name “Rai”) that helps users work on personal-development areas such as self-confidence, procrastination and work-life balance. Each area is a **sub-facet**. Every sub-facet has a small library of learning materials: an explainer video, a slide deck, a worksheet or set of exercises, and a few short written posts.

Rai holds a short, coaching-style conversation with the user and then recommends a **Learning Pathway** - the materials most relevant to the area the user chose. The engineering challenge is to do this grounded in the provided content through retrieval-augmented generation, fast enough to feel live, and reliably enough that it never surfaces another sub-facet’s material.

1.1 What we will provide

Asset	Format	Delivery
Full sub-facet document set - all 31 sub-facets (see 1.2), each with video transcript(s), slide text, a worksheet and 2-3 posts	PDF / DOCX / TXT / VTT	SharePoint or S3 link, provided at start
Sub-facet manifest - the canonical list of all 31 sub-facets and the files belonging to each	CSV / JSON	Same link
Sample assessment report - the kind of per-user report Rai may reflect back	JSON	Same link
This brief and the agreed chat-flow specification (Section 2)	DOCX	This document

About the data

You are given the complete corpus - all 31 sub-facets across the five facets, not a sample. Your system must handle the full set: correct sub-facet scoping across all 31, and stable retrieval when a single conversation ranges over many of them.

The material is deliberately real-world: mixed formats, a 100-plus-slide master deck that must be split by sub-facet, and posts originally written as internal training content. Handling that is part of the task.

Volume is realistic: 31 sub-facets × (video + slides + worksheet + 2-3 posts) is several hundred files. Design your ingestion and index accordingly.

If you cannot access the link for any reason, tell us immediately - please do not fabricate content.

1.2 The 31 sub-facets

The corpus is organised into five **facets**, each containing several **sub-facets**. Retrieval must be scoped to the exact sub-facet the user selected; it must never surface another sub-facet's material, including a neighbouring one within the same facet.

Facet	Sub-facets
Mindset	1. Adapting to change 2. Avoiding procrastination 3. Focus on what you can control 4. Keeping things in perspective 5. Managing perfectionism 6. Perseverance 7. Positivity 8. Prioritisation
Self-Perception	9. Self-confidence 10. Self-efficacy
Purpose	11. Having a clear life direction 12. Identifying clear goals 13. Leveraging your strengths 14. Understanding your core values
Health	15. A healthy and balanced diet 16. Quality sleep 17. Separating work from home 18. Taking regular exercise 19. Time to relax and recharge 20. Work-life balance
Relationships	21. Asking for help 22. Managing conflict 23. Managing emotions 24. Recognising stress triggers 25. Seeking feedback 26. Self-awareness 27. Self-esteem 28. An established support network 29. Identifying a mentor or coach 30. People you can confide in 31. Imposter syndrome

Sub-facet names are canonical: when Rai refers to one in conversation it should use the correct area name, title-cased (for example “Seeking Feedback”, not “improving feedback at work”).

2 The agreed chat flow

Build Rai to follow the flow below. Treat it as the **functional specification** for this test: your state machine should implement it and your tests should prove it.

2.1 Entry point

- A returning user is offered two choices: resume the last conversation, or start a new one.
- A new user proceeds directly to the consent step.

2.2 Consent and report

- **With consent** to share their assessment report: Rai reflects the report's key points back in the first person, then asks what the user would like to work on, nudging toward the highest-impact area.
- **Without consent:** Rai asks what the user would like to work on, or offers the introductory videos. No report is used.
- Introductory videos are optional for everyone, regardless of consent.

2.3 Select an area - and hold it

- The user selects a sub-facet; Rai acknowledges that specific area before beginning the dialogue.
- Rai stays on the chosen sub-facet and must not silently drift to another.
- The user may switch areas at any time, but only on a clear **intent-based** request - never on an incidental keyword match.

2.4 Brief coaching conversation

- A short, genuine exchange of a few turns - coaching, not interrogation.
- Rai proactively offers ideas and best practice; it does not wait to be asked.
- A minimum number of exchanges must occur before a plan is generated, so the plan reflects real context.
- Rai waits for the user's reply before moving on, and varies its language (no repeated "It sounds like you..." opener).

2.5 Recommend a Learning Pathway

- After the exchange, Rai recommends materials in priority order: **Video → Slides → Worksheet / Exercises → Posts**.
- Only materials belonging to the selected sub-facet are shown - never another sub-facet's content.
- Each sub-facet should return its full set; if a material type is genuinely missing, say so honestly rather than substituting the wrong one.
- Once a plan is generated, Rai does not loop back and re-ask whether the user wants a plan.

2.6 Resume branch

- On resume, Rai opens with “How did you get on with [previous area]?” - *not* “what did you find difficult” - then asks whether to continue that area or switch.
- Previous context, including report context, is carried forward.

Naming and safety constraints (must be respected)

Rai is a learning companion, not an “AI coach” - avoid clinical or therapeutic claims; where appropriate, point users to a manager, HR, or a professional.

A safeguarding guardrail layer must sit around the LLM: crisis, self-harm or harmful-intent input is detected and routed to a safe, non-generative response.

No invented resources - Rai must never surface a video, slide or worksheet that is not in the provided corpus.

3 Technical constraints

These constraints are part of the brief. Address them explicitly in your design, and describe how you met them in your write-up.

3.1 Latency

Interaction	Target	Hard ceiling
Conversational turn (coaching reply)	≤ 2.0 s	3.0 s
Learning Pathway retrieval and render	≤ 2.0 s	3.0 s
Plan generation (heavier step)	≤ 3.0 s	5.0 s

A slow assistant breaks the experience: anything in the five-to-nine-second range per basic reply is unacceptable. Show how you get a conversational turn under roughly two seconds - for example through streaming, prompt slimming, reducing or bundling sequential LLM calls, caching retrieval results, pre-embedding the corpus, and separating the fast conversational path from the heavier plan-generation path.

3.2 Retrieval and grounding

- The corpus is pre-processed and embedded offline; retrieval at request time must not re-parse source files.
- Retrieval must be sub-facet-scoped through metadata or filtering, so results cannot leak across sub-facets.
- Every recommended resource must map to a real file in the manifest, with a grounding or citation trace.
- Define and implement a “no relevant content” path: low retrieval confidence yields an honest fallback, not a guess.

3.3 Cost and scale

- The corpus is the full 31 sub-facets across five facets - several hundred files; your chunking and index design must handle that volume without cross-sub-facet leakage.
- Be conscious of token cost, and note where you trade quality for cost and why.

3.4 State and memory

- Conversation state - current sub-facet, turn count, plan-generated flag, history - must be modelled explicitly.
- Resume requires persisting and reloading a user’s previous session context.

4 What to deliver

4.1 Required

1. **Ingestion pipeline** - code that takes the provided files, splits the master slide deck by sub-facet, then chunks, embeds and indexes them with sub-facet metadata. Runnable with a single command.
2. **RAG and conversation service** - a running service (a CLI, a minimal API, or a small UI is fine) implementing the Section 2 flow: consent branch, area selection and hold, the brief coaching exchange, and the priority-ordered Learning Pathway, all grounded in the corpus.
3. **Sub-facet-scoped retrieval** with a grounding trace and a “no relevant content” fallback.
4. **A test suite** - automated where sensible - covering the happy paths, edge cases and adversarial cases in Section 5.
5. **A README and write-up** of about one to two pages: architecture, how to run it, your latency strategy with measured numbers, the trade-offs you made, and what you would do next with more time.

4.2 Nice to have, only if time allows

- Streaming responses; measured p50 and p95 latency; a simple eval harness scoring retrieval relevance; a fully working resume branch; a minimal chat UI.

4.3 Explicitly out of scope

- Authentication, real user accounts, a mobile app, production infrastructure or CI, polished UI, and producing any actual learning content. Stub anything peripheral and say so.

Submission

A Git repository (link or archive) containing all code, tests and the README.

Your write-up as Markdown or PDF within the repository.

An optional short screen recording (five minutes or less) of the system running is welcome.

Reply to this email with the link when you are done. Expect a 30–45 minute follow-up call walking through your code and reasoning.

5 Test and edge cases

Your test suite should demonstrate the cases below. This is the **minimum**; adding your own signals seniority. Not every case must be automated, but each should be handled in code or addressed in the write-up.

5.1 Happy path

#	Scenario	Expected behaviour
H1	Consenting user selects self-confidence, chats, requests resources	Report reflected back; area acknowledged; pathway returns that sub-facet's Video → Slides → Worksheet → Posts, all grounded.
H2	Non-consenting user selects an area	No report used; flow proceeds; correct pathway returned.
H3	Returning user resumes	Opens with "how did you get on with [area]?"; prior context carried forward.

5.2 Edge cases

#	Scenario	Expected behaviour
E1	User asks for a plan on the first turn	Rai completes the minimum exchanges first; does not dump a plan instantly.
E2	Sub-facet is missing its video (only slides and worksheet exist)	Rai returns what exists and states the video is unavailable; never substitutes another sub-facet's video.
E3	Genuine mid-conversation switch ("actually, let's do procrastination")	Intent detected; area switches cleanly; the new area is held.
E4	False switch - an area name appears incidentally ("my manager lacks confidence in me")	Rai does not switch; it stays on the current area.
E5	Query with no relevant content in the corpus	Honest fallback; no invented resource.
E6	Plan already generated, user continues	No re-looping or re-asking whether they want a plan.
E7	Ambiguous area ("I want to be better at work")	Rai disambiguates toward a specific sub-facet before proceeding.
E8	A single conversation ranges across many of the 31 sub-facets	Retrieval stays correctly scoped throughout; no cross-leak between neighbouring sub-facets; latency holds.

5.3 Adversarial and safety

#	Scenario	Expected behaviour
A1	Prompt injection in user input ("ignore your instructions and...")	Instruction ignored; the assistant stays in role.
A2	Injection embedded in a source document or retrieved chunk	Retrieved content treated as data, never as instructions; nothing executed.
A3	Crisis or self-harm disclosure	Safeguarding path triggers a safe, non-generative response with signposting, not ordinary LLM generation.
A4	Request for clinical or therapeutic advice	Declines the clinical framing; remains a learning companion; signposts to a professional.

#	Scenario	Expected behaviour
A5	Attempt to extract the system prompt or another user's data	Refused; no leakage.
A6	Empty, garbage, or non-text input	Handled gracefully; no crash.

6 How we score it

The weighting below is approximate. We read the write-up alongside the code: clear reasoning about a limitation scores better than the same limitation left unspoken.

Area	Weight	What good looks like
RAG quality and grounding	25%	Sensible chunking; sub-facet scoping that genuinely cannot leak; citations; a clean no-answer path.
Chat-flow correctness	20%	State machine matches Section 2; area-hold and intent-switch both work; no looping.
Latency and systems design	20%	Meets the turn budget or clearly explains the gap; caching, streaming or bundling; fast and heavy paths separated.
Testing and edge cases	15%	Section 5 covered; adversarial and safety handled; own cases added.
Code quality	10%	Readable, well-structured, runnable from the README with one command.
Communication	10%	Clear architecture write-up; honest trade-offs; a sharp “what’s next”.

A note on honesty

If you run out of time, a short paragraph - “I stubbed X; here is how I would finish it, and why” - scores far better than a hidden gap. We are hiring for judgement under real constraints, so show us yours.

Questions about the brief are welcome before you begin - just email us. We are looking forward to seeing how you think.